Punch Verification API Documentation

Base URL: `https://aiimagedetection.onrender.com` (or your local development URL like `http://127.0.0.1:8000`)

Version covering the simplified two-status flow (approved / rejected) with service-error reporting, per-photo reason summaries and service failure details.

General Information

- All POST requests must use `Content-Type: application/json`.
- Images must be JPG or PNG, sent as Base64 strings. Raw Base64 is recommended; a data URL prefix is also accepted.
- PDFs are accepted in the request but are not verified. Only uploaded images are checked.
- Only two statuses are ever returned, for each photo and for the whole request: `approved` and `rejected`.
- Every image is verified on its own. If any one image is rejected, the whole request is rejected.
- The response always has the same shape: `status, error, service_error, service_errors, results`.

1. Check Server Health

Returns the connection status of the AI and verification services.

Endpoint:

GET **`/health`**

Response (200 OK):

```
{
  "status": "healthy",
  "sightengine_keys_loaded": true,
  "gemini_api_loaded": true,
  "c2pa_loaded": true
}
```

These flags only show that keys or clients are loaded. They do not prove the keys are still valid.

2. Get Office Locations

Fetches the list of available offices and their coordinates for the UI dropdown.

Endpoint:

GET **`/api/offices`**

Response (200 OK):

```
[
  {
    "name": "Guindy Office",
    "lat": "13.0067",
    "lng": "80.2206"
  },
  {
    "name": "Vadapalani Office",
    "lat": "13.0503",
    "lng": "80.2121"
  }
]
```

3. Verify Punch Request (Primary Endpoint)

Verifies the GPS camera photos, checks metadata and credentials for editing or AI generation, and validates the photo date, time and location against the requested attendance date, times and office.

Endpoint:

POST **`/punch-verify`**

Headers:

- `Content-Type: application/json`
- `Authorization: Bearer <token>` (Optional, if enabled)

Input Format (JSON)

```
{
  "images": [
    {
      "image_base64": "iVBORw0KGgoAAAANSUhEUgAA... (very long string)",
      "file_name": "photo1.jpg"
    }
  ],
  "pdfs": [
    {
      "pdf_base64": "JVBERi0xLjQKJcfsj6IK... (very long string)",
      "file_name": "document.pdf"
    }
  ],
  "adjustment_type": 0,
  "attendance_date": "2026-11-05",
  "expected_times": ["09:30", "18:30"],
  "office_latitude": 13.0067,
  "office_longitude": 80.2206
}
```

Field Definitions:

Field	Required	Description
`images`	No*	Array of image objects (**image_base64**, **file_name**). Can be empty if pdfs has files. A single image can also be sent as **image_base64** at the top level, used when images is empty.
`pdfs`	No	Array of PDF objects (**pdf_base64**, **file_name**). PDFs are ignored. A request with only PDFs and no images is approved with an empty results list.

Field	Required	Description
`adjustment_type`	Yes	Integer 0 to 3 (see the next table). Any other value is rejected with 'The adjustment type is not recognized.'
`attendance_date`	Recommended	Format YYYY-MM-DD. The photo date must equal this date.
`expected_times`	Recommended	Array of 24-hour times, HH:MM or HH:MM:SS. The photo time, to the minute, must match one of them.
`office_latitude` / `office_longitude`	Recommended	GPS coordinates of the selected office. If missing, the distance check is skipped and a note is added (the photo is not rejected for it).

adjustment_type

Value	Meaning	Notes
0	Regular adjustment (proof image)	Each uploaded photo is verified.
1	Static on-site	Normally two photos (check-in and check-out). With 2 or more photos, more than one expected time, and every photo matched to a time, all expected times must appear across the photos.
2	Semi punch	Each uploaded photo is verified.
3	Check-in correction	Each uploaded photo is verified.

The API no longer returns an 'invalid' status for the number of photos. Photo counts are not enforced; every uploaded image is verified.

How a photo is verified

Stage	Check	Result
A. Metadata scan	Image info chunks, EXIF (Software, HostComputer, ImageDescription, Artist) and XMP are scanned for known editor and AI-generator signatures.	Unreadable file: rejected. A signature hit marks the photo as edited (rejected at stage B).
B. C2PA credentials	Content credentials are inspected.	AI-generated or edited credentials, or a metadata signature: rejected. Flags only: kept as notes and the photo continues to stage C.
C. Gemini vision	Reads GPS, date and time from the photo and gives AI/edit verdicts with confidence scores.	Gemini failure: rejected as a service error.
D. Rule checks	Photo date, photo time and distance from the office are compared with the request.	See the decision rules below.
E. Sightengine	Independent AI-image score.	Score at or above the threshold: rejected. Sightengine failure: rejected as a service error.

Decision rules (stage D, in this order)

- **Rejected** if the photo date does not match `attendance_date`, the photo time does not match any expected time, or the photo is farther than the allowed office distance.
- **Rejected** if Gemini marks the photo as edited with confidence at or above the edit-reject threshold.
- **Rejected** if the GPS or clock could not be read and Gemini's missing-data confidence is at or above the missing-reject threshold.
- **Approved with notes** (Sightengine is skipped) if there are softer problems: GPS or clock unreadable at low confidence, no office coordinates, or an edit verdict that needs review. The notes are in the photo's `error` field.
- Otherwise the photo goes to Sightengine. A score at or above the threshold is rejected. Below it the photo is approved; if Gemini said AI-generated or gave no edit verdict, a note is added.

The numeric limits (office radius, confidence thresholds, Sightengine threshold) are server settings: `OFFICE_RADIUS_METRES`, `GEMINI_EDIT_REJECT_CONFIDENCE`, `GEMINI_EDIT_REVIEW_CONFIDENCE`, `GEMINI_MISSING_REJECT_CONFIDENCE` and `SIGHTENGINE_AI_THRESHOLD`.

Output Format (JSON)

Top-level fields:

Field	Type	Description
`status`	string	'approved' or 'rejected'.
`error`	string or null	Why the request was rejected. Null when approved. If any photo was rejected for a real reason, only the real reasons are listed here. With several images, each message starts with the file name.
`service_error`	boolean	True only when every rejection was caused by a service failure. False if any photo was rejected for a real reason, and when nothing was rejected.
`service_errors`	array	Photos that failed because of a service problem, as {file_name, service, error, detail}. Always filled when such failures happen, even if service_error is false. Empty array otherwise. See the service_errors fields below.
`results`	array	One entry per uploaded image, in upload order. Empty when no image was verified.

Per-photo fields (results[]):

Field	Description
`file_name`	File name sent in the request, if any.
`status`	'approved' or 'rejected'.
`reason`	A concise summary of why the photo was rejected (for example 'Marked as AI generated, Date mismatch'). Null when there is nothing to report.
`metadata_edited`, `metadata_flags`	Editor or generator signatures found in metadata or C2PA; flags also hold C2PA notes.

Field	Description
`c2pa_present`, `c2pa_state`, `c2pa_issuer`	C2PA content credential details.
`sightengine_score`, `sightengine_is_ai`	Sightengine AI score and verdict. Null if the photo never reached Sightengine.
`is_ai_generated`, `is_edited`	Gemini verdicts.
`edit_confidence`, `missing_confidence`	Gemini confidence values between 0 and 1.
`latitude`, `longitude`, `timestamp`, `location`, `accuracy`	Values read from the photo's GPS camera overlay (as text).
`distance_metres`	Distance from the office in metres, when GPS and office coordinates are available.
`time_match`	True when both the photo date and time match the request.
`matched_minute`	The expected time (HH:MM) the photo matched. Only set when the photo passed every check.
`service_error`	True when this photo was rejected because a service failed, not because of the photo.
`service_name`	Which part failed: gemini, sightengine or internal. Set only when service_error is true.
`service_error_detail`	Actual cause of the service failure, with secrets removed.
`error`	Reason for rejection, or notes on an approved photo. Null when there is nothing to report.

service_errors[] entries:

Field	Description
`file_name`	File name of the photo that failed, if any.
`service`	Which part failed: gemini, sightengine or internal.
`error`	Generic message shown to users.
`detail`	Actual cause of the failure, secrets removed, maximum 300 characters.

Scenario A: Approved ("status": "approved")


```
{
  "status": "approved",
  "error": null,
  "service_error": false,
  "service_errors": [],
  "results": [
    {
      "file_name": "photo1.jpg",
      "status": "approved",
      "metadata_edited": false,
      "metadata_flags": [],
      "c2pa_present": false,
      "c2pa_state": null,
      "c2pa_issuer": null,
      "sightengine_score": 0.01,
      "sightengine_is_ai": false,
      "is_ai_generated": false,
      "is_edited": false,
      "edit_confidence": 0.02,
      "missing_confidence": 0.0,
      "latitude": "13.006575",
      "longitude": "80.206510",
      "timestamp": "05/11/2026 09:30:15 AM",
      "location": "Guindy, Chennai, Tamil Nadu, India",
      "accuracy": "+/-10.0 m",
      "distance_metres": 12.5,
      "time_match": true,
      "matched_minute": "09:30",
      "service_error": false,
      "error": null
    }
  ]
}
```


An approved photo can still carry notes in its own **error** field (for example C2PA flags, or an unreadable GPS at low confidence). The request-level **error** stays null when approved.

Scenario B: Rejected (real reason)

Returned if the photo is AI-generated, edited, has the wrong date or time, or is outside the allowed distance from the office.


```
{
  "status": "rejected",
  "error": "The photo date 04/11/2026 09:30 AM does not match the
            attendance date 05/11/2026.",
  "service_error": false,
  "service_errors": [],
  "results": [
    {
      "file_name": "photo1.jpg",
      "status": "rejected",
      "reason": "Date mismatch",
      "time_match": false,
      "service_error": false,
      "error": "The photo date 04/11/2026 09:30 AM does not match the
                attendance date 05/11/2026.",
      "...": "..."
    }
  ]
}
```

Scenario C: Service failure (every rejection)

Returned when Gemini or Sightengine could not be used (for example an expired key, quota or network error) or an unexpected internal error happened. The photo is not at fault; ask the user to try again later. Show users the generic message in **error**; the actual cause (secrets removed) is in **service_error_detail** on the photo and **detail** in **service_errors**, for your own logs and debugging.


```
{
  "status": "rejected",
  "error": "Photo verification is temporarily unavailable. Please try again later.",
  "service_error": true,
  "service_errors": [
    {
      "file_name": "photo1.jpg",
      "service": "gemini",
      "error": "Photo verification is temporarily unavailable. Please try again later.",
      "detail": "Gemini API key expired"
    }
  ],
  "results": [
    {
      "file_name": "photo1.jpg",
      "status": "rejected",
      "service_error": true,
      "service_name": "gemini",
      "service_error_detail": "Gemini API key expired",
      "error": "Photo verification is temporarily unavailable. Please try again later.",
      "...": "..."
    }
  ]
}
```


Scenario D: Mixed (real rejection and a service failure)

Two photos: a.jpg has the wrong time, and the service failed for b.jpg. Because a photo was rejected for a real reason, **service_error** is false and **error** lists only the real reason. The service failure is reported in **service_errors**.


```
{
  "status": "rejected",
  "error": "a.jpg: The photo time 08:00 does not match 09:30.",
  "service_error": false,
  "service_errors": [
    {
      "file_name": "b.jpg",
      "service": "sightengine",
      "error": "b.jpg: Photo verification is temporarily unavailable. Please try again later.",
      "detail": "Sightengine request timed out"
    }
  ],
  "results": [
    { "file_name": "a.jpg", "...": "..." },
    { "file_name": "b.jpg", "service_error": true, "service_name": "sightengine", "...": "..." }
  ]
}
```

Scenario E: Static on-site, a required time is missing

```
{
  "status": "rejected",
  "error": "Both the check-in time and the check-out time must appear on the
            photos. Missing 18:30.",
  "service_error": false,
  "service_errors": [],
  "results": [ "..." ]
}
```

Scenario F: PDF only

PDFs are not verified. A request with PDFs and no images is approved with an empty results list.

```
{
  "status": "approved",
  "error": null,
  "service_error": false,
  "service_errors": [],
  "results": []
}
```

Scenario G: No photo, or unknown adjustment type

```
{
  "status": "rejected",
  "error": "No photo was uploaded.",
  "service_error": false,
  "service_errors": [],
  "results": []
}
```

```
{
  "status": "rejected",
  "error": "The adjustment type is not recognized.",
  "service_error": false,
  "service_errors": [],
  "results": []
}
```

4. Verify Punch Request (URLs)

If your images are already hosted on a public server (such as AWS S3 or Cloudinary), pass the URLs instead of Base64 strings.

Endpoint:

POST **`/punch-verify-urls`**

Input Format (JSON)

```
{
  "images": [
    {
      "image_url": "https://your-bucket.s3.amazonaws.com/photo1.jpg",
      "file_name": "photo1.jpg"
    }
  ],
  "adjustment_type": 1,
  "attendance_date": "2026-11-05",
  "expected_times": ["09:30", "18:30"],
  "office_latitude": 13.0067,
  "office_longitude": 80.2206
}
```

The URL must be a full http or https link to a JPG or PNG. If an image cannot be downloaded, that photo is rejected with the download error. The output format is identical to `/punch-verify`.

5. Error Message Reference

Message (photo or request)	Meaning
The adjustment type is not recognized.	adjustment_type is not 0, 1, 2 or 3.
No photo was uploaded.	No images and no PDFs were sent.
The image could not be read.	Empty or invalid Base64, or the file could not be stored.
The file is not a readable image. / Image metadata could not be read.	The file is not a valid image or its metadata is unreadable.
The photo credentials show it is AI-generated: ...	C2PA credentials say the photo is AI-generated.
The photo metadata shows it was edited: ...	An editor or generator signature was found (metadata or C2PA).
The photo date ... does not match the attendance date ...	Photo date differs from attendance_date.
The photo time ... does not match ...	Photo time is not one of expected_times.
The photo is ... metres from the office. The allowed distance is ... metres.	Photo is outside the allowed office radius.
The photo has no readable GPS address. / The photo clock could not be read.	Rejected only when Gemini's missing-data confidence is high; otherwise a note on an approved photo.
This branch has no office coordinates.	office_latitude/office_longitude were not sent; the distance check was skipped (note only).
Gemini marked the photo as edited. / ... as AI-generated.	Gemini verdicts; they reject only under the confidence rules above.
Sightengine marked the photo as AI-generated.	Sightengine score is at or above the threshold.
Both the check-in time and the check-out time must appear on the photos. Missing ...	Static on-site: an expected time is not covered by any photo.
Photo verification is temporarily unavailable. Please try again later.	Service failure (service_error is true on the photo).

6. Changes From the Previous Version

Before	Now
Statuses real, pending, rejected, invalid.	Only approved and rejected. Old 'real' and 'pending' both become 'approved'. 'invalid' is removed.
A PDF returned 'pending' for manual review.	PDFs are ignored. A PDF-only request is approved; images are always verified.
Exact photo count enforced per adjustment type.	Photo counts are not enforced. Each image is verified, and any rejected image rejects the request.
C2PA flags could end the check early.	C2PA flags are notes only; the photo continues through every later check.
API failures could surface as errors or 500 responses.	Gemini and Sightengine failures are rejected with service_error = true and a generic message.
Response had status, error, results.	Response also has service_error and service_errors. Each photo has service_error.
Service failure cause was only written to the server log.	Each photo carries service_name and service_error_detail; each service_errors entry carries service and detail (secrets removed, max 300 characters).
Rejected photos had only an error message.	Each photo also has a short reason summary, for example 'Marked as AI generated, Date mismatch'.